Introduction to Cognitive Science

HomeWork #2

Sina Pirmoradian : 810101125

HW #2 Part #1

Connect Google Drive

```
from google.colab import drive
drive.mount ('/content/drive')
```

```
Mounted at /content/drive
```

Import Reqiured Library

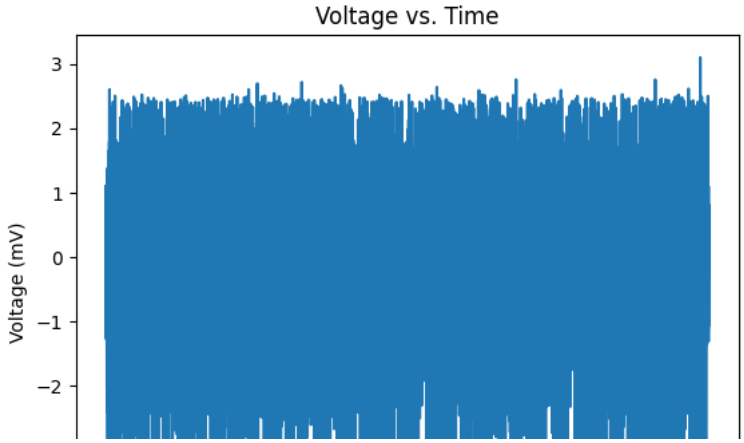
```
import numpy as np
from scipy import signal
import scipy.io as sio
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.manifold import TSNE
```

Loading Data

```
# Load the data from the .mat file
data = sio.loadmat('/content/drive/MyDrive/Cognetive/HW_2/extracellular.mat')

spike = sio.loadmat('/content/drive/MyDrive/Cognetive/HW_2/spikes.mat')
spike_idx = spike['SpikeInds'][0]
```

Extracextract the Voltage Amplitude

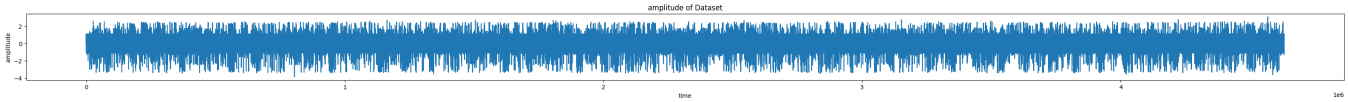
```
# extract the voltage amplitude data from the structure
voltage = data['all_data_with_noise_and_line'][0,:]

# Sampling rate of the recording is 2400Hz
sampling_rate = 2400

# Create a time axis based on the sampling rate of 2400Hz
sampling_rate = 2400
time = np.arange(len(voltage)) / sampling_rate

# Plot the voltage against time
plt.plot(time, voltage)
plt.xlabel('Time (s)')
plt.ylabel('Voltage (mV)')
plt.title('Voltage vs. Time')
plt.show()

```



Plot the voltage and against time in a wide shape

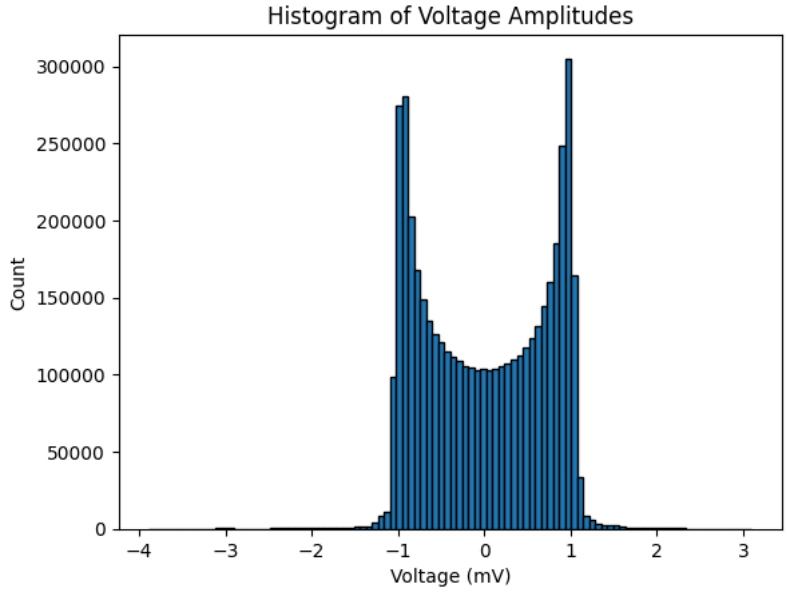
```
1 # Plot the voltage against time
2 plt.figure(figsize=(40,2))
3 plt.plot(voltage)
4 plt.title('amplitude of Dataset')
5 plt.xlabel('time')
6 plt.ylabel('amplitude')
```

```
Text(0, 0.5, 'amplitude')
```



Plot the histogram of the voltage amplitudes

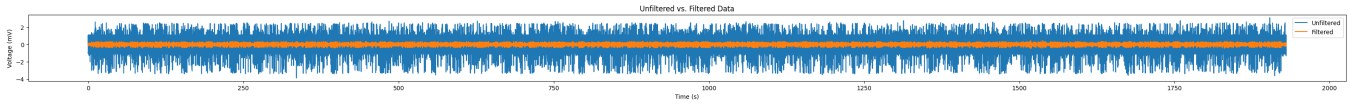
```
1 # Flatten the voltage array to a 1D array
2 voltage_flat = voltage.flatten()
3
4 # Plot the histogram of the voltage amplitudes
5 plt.hist(voltage_flat, bins=100, edgecolor='black')
6 plt.xlabel('Voltage (mV)')
7 plt.ylabel('Count')
8 plt.title('Histogram of Voltage Amplitudes')
9 plt.show()
```



Plot Unfiltered vs. Filtered Data Together

```
1 # Define filter parameters
2 highpass_freq = 300
```

```
filter_order = 7

# Calculate the Nyquist frequency
nyquist_freq = sampling_rate / 2

# Calculate the highpass cutoff frequency as a fraction of the Nyquist frequency
highpass_cutoff = highpass_freq / nyquist_freq

# Design the highpass Butterworth filter
b, a = signal.butter(filter_order, highpass_cutoff, btype='highpass')

# Apply the filter to the data using the filtfilt function
filtered_voltage = signal.filtfilt(b, a, voltage)

# Create a time axis based on the number of samples and sampling rate
time = np.arange(len(voltage)) / sampling_rate

plt.figure(figsize=(40,2))
# Plot the unfiltered data
plt.plot(time, voltage, label='Unfiltered')
# Plot the filtered data
plt.plot(time, filtered_voltage, label='Filtered')
plt.xlabel('Time (s)')
plt.ylabel('Voltage (mV)')
plt.title('Unfiltered vs. Filtered Data')
plt.legend()
plt.show()

```

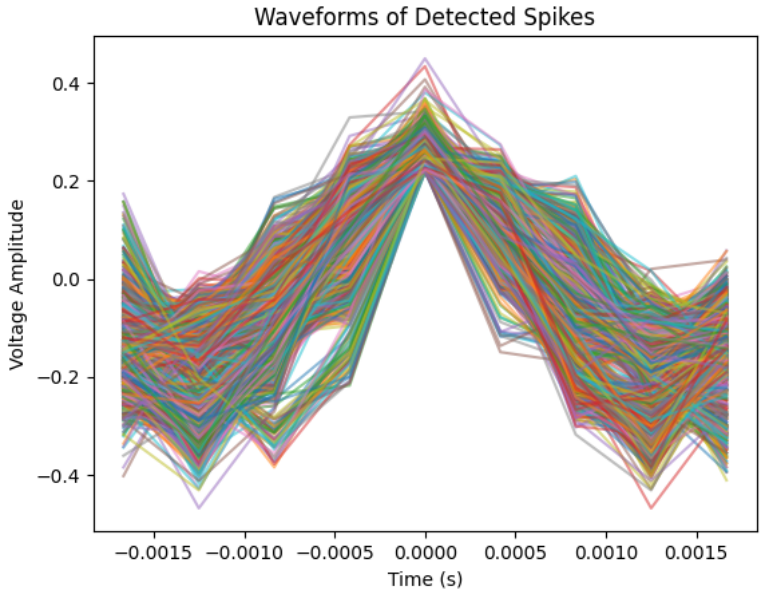


Filtering: Applying a Bandpass Filter

```
# extract the voltage amplitude data from the structure
voltage = data['all_data_with_noise_and_line'][0,:]

# Define filter parameters
fs = 2400  # Sampling rate
highpass_freq = 300
filter_order = 7

# Calculate the Nyquist frequency
sampling_rate = 2400
nyquist_freq = sampling_rate / 2

# Calculate the highpass cutoff frequency as a fraction of the Nyquist frequency
highpass_cutoff = highpass_freq / nyquist_freq

# Design the highpass Butterworth filter
b, a = signal.butter(filter_order, highpass_cutoff, btype='highpass')

# Apply the filter to the data using the filtfilt function
filtered_voltage = signal.filtfilt(b, a, voltage)

# Calculate the voltage threshold (θ)
sigma_n = np.median(np.abs(filtered_voltage)) / 0.6745
threshold = 5 * sigma_n

print("Voltage Threshold (θ):", threshold)

# Extract peaks in the data
peaks, _ = find_peaks(filtered_voltage, height=0)  # Find peaks with positive height

# Select points above the threshold
spike_points = peaks[filtered_voltage[peaks] >= threshold]

# Define the time window for waveform extraction
window_size = int(fs * 0.002)  # 2ms before and after the peak
time_window = np.arange(-window_size, window_size + 1) / fs

# Extract waveforms for each detected spike
waveforms = []
for point in spike_points:
    waveform = filtered_voltage[point - window_size: point + window_size + 1]
    waveforms.append(waveform)
```

```
43
44 waveforms = np.array(waveforms)
45
46 # Plot the waveforms for each detected spike
47 plt.figure()
48 for waveform in waveforms:
49     plt.plot(time_window, waveform, alpha=0.5)
50
51 plt.xlabel('Time (s)')
52 plt.ylabel('Voltage Amplitude')
53 plt.title('Waveforms of Detected Spikes')
54 plt.show()
55
56 # Store the waveforms in a 2D matrix for the next step
57 # waveforms is the 2D matrix containing the waveforms
58
```

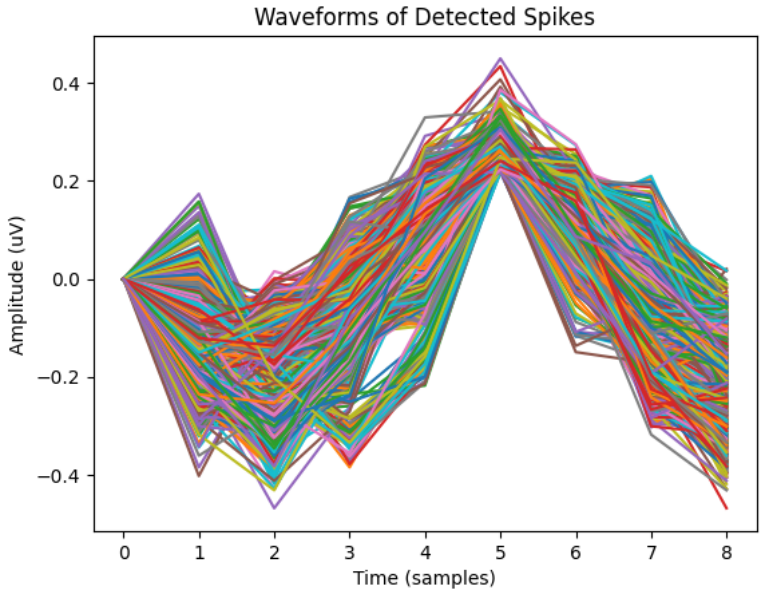
```
Voltage Threshold (θ): 0.2165574093077027
```



Spike Detection

```
 1 # Define high-pass filter parameters
 2 fs = 2400  # Sampling rate
 3 f_high = 300  # High-pass cutoff frequency
 4 order = 7  # Filter order
 5
 6 # Design the filter coefficients
 7 b, a = signal.butter(order, f_high / (fs / 2), btype='highpass')
 8
 9 # Apply the filter to the data
10 filtered_data = signal.filtfilt(b, a, voltage)
11
12 # Calculate the voltage threshold
13 sigma_n = np.median(np.abs(filtered_data)) / 0.6745
14 theta = 5 * sigma_n
15
16 # Find peaks in the data
17 diff_data = np.diff(filtered_data)
18 peaks = np.where((diff_data[:-1] > 0) & (diff_data[1:] < 0))[0] + 1
19
20 # Find spikes based on threshold
21 spikes = peaks[filtered_data[peaks] >= theta]
22
23 # Extract waveform for each spike
24 window_size = int(0.004 * fs)  # 2ms window
25 waveforms = np.zeros((len(spikes), window_size))
26 for i, s in enumerate(spikes):
27     waveform_start = max(s - window_size // 2, 0)
28     waveform_end = min(s + window_size // 2, len(filtered_data))
29     waveform = filtered_data[waveform_start:waveform_end]
30     if len(waveform) < window_size:
31         waveform = np.concatenate((np.zeros(window_size - len(waveform)), waveform))
32     waveforms[i, :] = waveform
33
34 # Plot waveforms
35 plt.figure()
```

```
plt.plot(waveforms.T)
plt.xlabel('Time (samples)')
plt.ylabel('Amplitude (uV)')
plt.title('Waveforms of Detected Spikes')
plt.show()
waveforms = np.array(waveforms)
```



Feature extraction using PCA

```
# Apply PCA on the waveform matrix
pca = PCA(n_components=8)
waveform_pca = pca.fit_transform(waveforms)
print(pca.singular_values_)

PC1 = waveform_pca[:, 0]
PC2 = waveform_pca[:, 1]
PC3 = waveform_pca[:, 2]
PC = [PC1, PC2, PC3]

pca_waveform = PC

pca_waveform = np.array(PC)
pca_waveform = np.transpose(pca_waveform)
```

```
[8.5529941  5.61293533 3.28665284 2.82368678 2.57025087 2.49363466
 1.84777402 0.72617792]
```

Clustering using K-Means Clustering using PCA

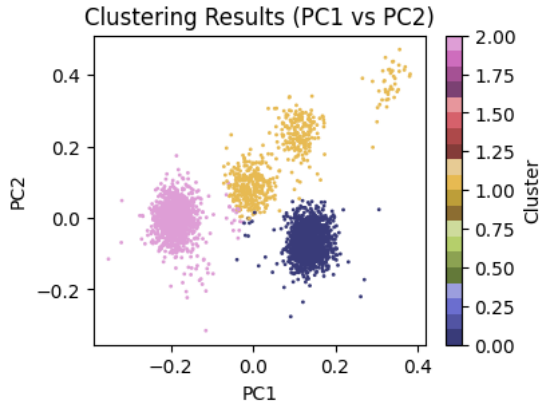
```
# Perform K-Means clustering
n_clusters = 3  # Number of clusters
kmeans = KMeans(n_clusters=n_clusters, n_init="auto")
kmeans.fit(pca_waveform)  # Use PC1, PC2, and PC3 as features for clustering

# Get the cluster labels for each waveform
cluster_labels = kmeans.labels_

# Print the cluster labels
print("Cluster Labels:", cluster_labels)
```

```
Cluster Labels: [2 0 2 ... 0 2 0]
```

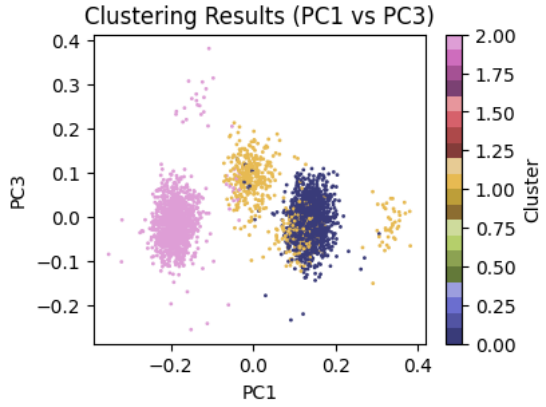
Plot Scatter plot of #3 main component principle

```
# Scatter plot for PC1 and PC2
plt.figure(figsize=(4, 3))
plt.scatter(PC1, PC2, c=cluster_labels, cmap='tab20b', s=1)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('Clustering Results (PC1 vs PC2)')
```

```
plt.colorbar(label='Cluster')
plt.show()
```

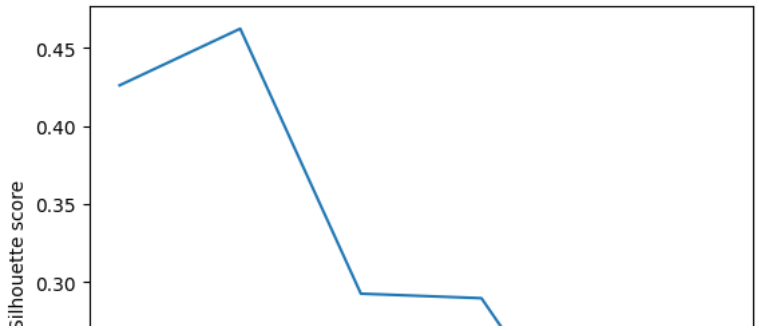



```
# Scatter plot for PC1 and PC3
plt.figure(figsize=(4, 3))
plt.scatter(PC1, PC3, c=cluster_labels, cmap='tab20b', s=1)
plt.xlabel('PC1')
plt.ylabel('PC3')
plt.title('Clustering Results (PC1 vs PC3)')
plt.colorbar(label='Cluster')
plt.show()
```



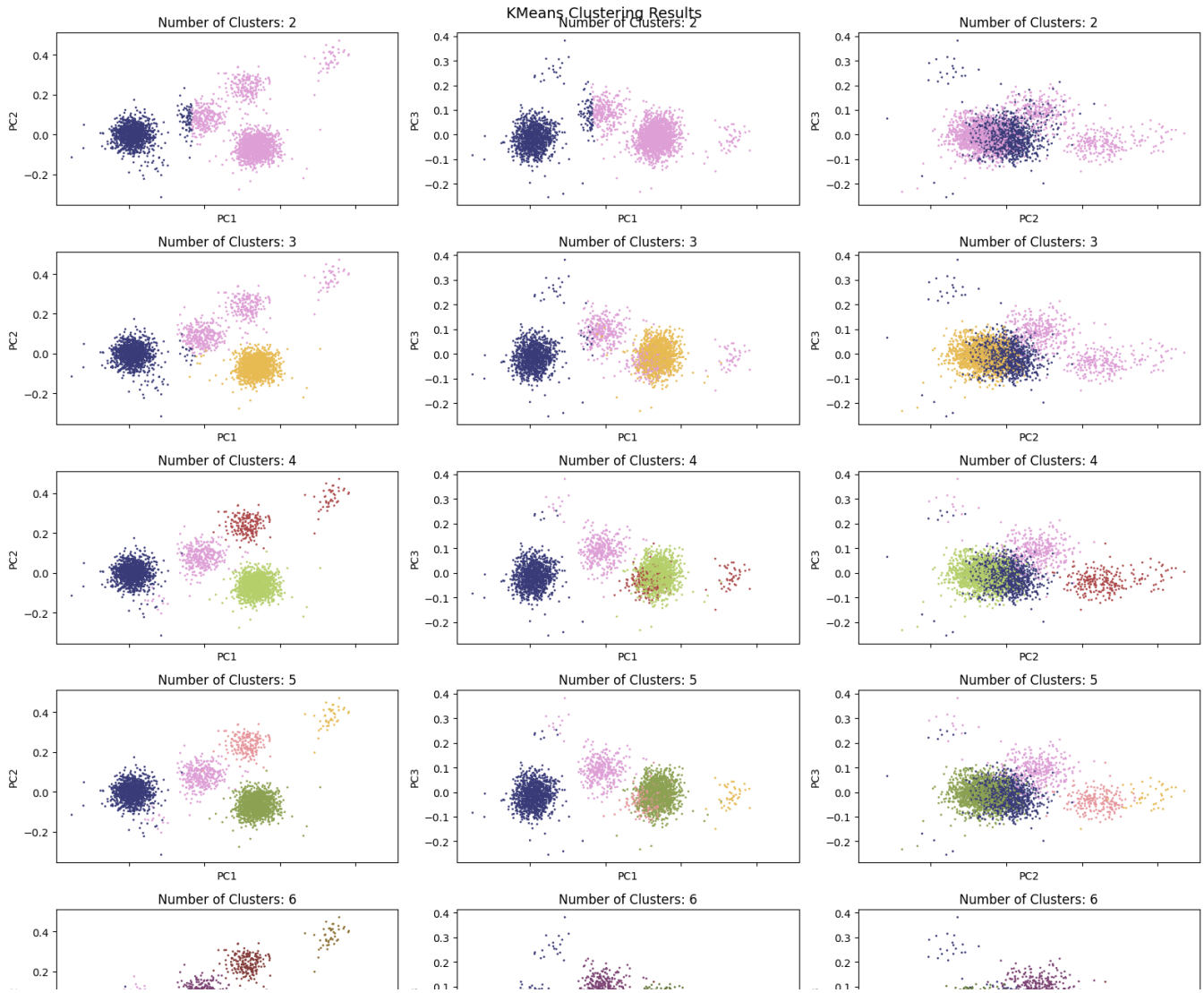
Using silhouette_score to Find a K which was the best performance using PCA

```
# Define a range of values for K
k_values = range(2, 8)

# Calculate silhouette scores for each value of K
silhouette_scores = []
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
    labels = kmeans.fit_predict(waveform_pca)
    score = silhouette_score(waveform_pca, labels)
    silhouette_scores.append(score)

# Plot the silhouette scores for each value of K
import matplotlib.pyplot as plt

plt.plot(k_values, silhouette_scores)
plt.xlabel('Number of clusters (K)')
plt.ylabel('Silhouette score')
plt.show()

```



Plot the Result of clustering with different K's using PCA

```
# Repeat part (k) and (l) with different values of K for the clustering algorithm
k = 8
k_values = range(2, k)
fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
for i in k_values:
    kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(waveform_pca)
    labels = kmeans.labels_
    axes[i-2, 0].scatter(waveform_pca[:,0], waveform_pca[:,1], c = labels, s=1, cmap='tab20b')
    axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 0].set_xlabel('PC1')
    axes[i-2, 0].set_ylabel('PC2')

    axes[i-2, 1].scatter(waveform_pca[:,0], waveform_pca[:,2], c = labels, s=1, cmap='tab20b')
    axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 1].set_xlabel('PC1')
    axes[i-2, 1].set_ylabel('PC3')

    axes[i-2, 2].scatter(waveform_pca[:,1], waveform_pca[:,2], c = labels, s=1, cmap='tab20b')
    axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 2].set_xlabel('PC2')
    axes[i-2, 2].set_ylabel('PC3')

# Add overall title and adjust layout
fig.suptitle('KMeans Clustering Results', fontsize=14)
fig.tight_layout()
```



Compute performance metrics of the K-means clustering by using PCA

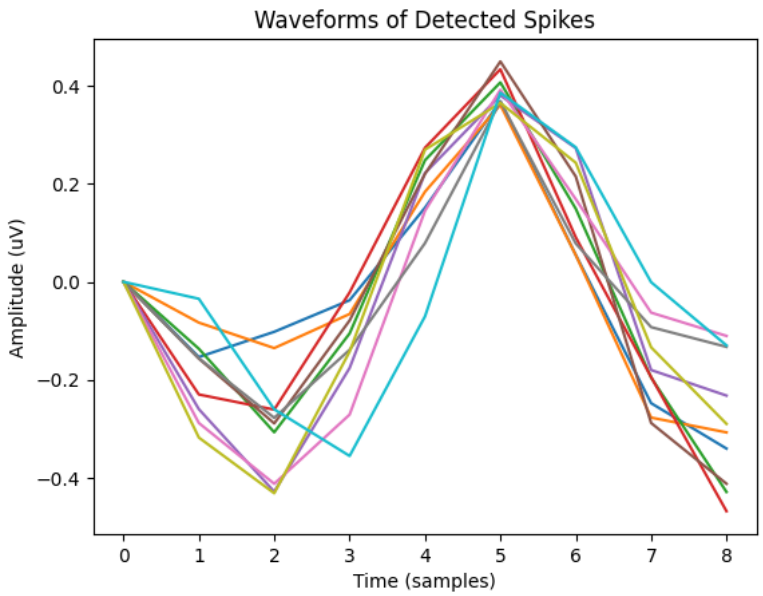
```
# Compute performance metrics
time_window = 0.001  # +/- 1ms
true_positives = 0
false_positives = 0
false_negatives = 0

for spike_time in spike_idx:
    if np.any(np.abs(peaks - spike_time) <= time_window * fs):
        true_positives += 1
    else:
        false_negatives += 1

for spike_time in spikes:
    if np.all(np.abs(spike_time - spike_idx) > time_window * fs):
        false_positives += 1

precision = true_positives / (true_positives + false_positives)
recall = true_positives / (true_positives + false_negatives)
f1_score = 2 * (precision * recall) / (precision + recall)

print("Precision: %.3f" % precision)
print("Recall: %.3f" % recall)
print("F1-Score: %.3f" % f1_score)

```

```
Precision: 0.772
Recall: 0.957
F1-Score: 0.855
```

Filtering: Applying a Bandpass Filter Using New Threshold

```
# extract the voltage amplitude data from the structure
voltage = data['all_data_with_noise_and_line'][0,:]

# Define high-pass filter parameters
fs = 2400  # Sampling rate
f_high = 300  # High-pass cutoff frequency
order = 7  # Filter order

# Design the filter coefficients
b, a = signal.butter(order, f_high / (fs / 2), btype='highpass')

# Apply the filter to the data
filtered_data = signal.filtfilt(b, a, voltage)

# Find peaks in the data
diff_data = np.diff(filtered_data)
peaks = np.where((diff_data[:-1] > 0) & (diff_data[1:] < 0))[0] + 1

# Calculate the voltage threshold
new_theta = 0.8 * np.max(filtered_data)
p = filtered_data[peaks] >= new_theta

# Find spikes based on threshold
new_spikes = peaks[filtered_data[peaks] >= new_theta]
print(new_spikes)

# Extract waveform for each spike
window_size = int(0.004 * fs)  # 2ms window
new_waveforms = np.zeros((len(new_spikes), window_size))
for i, s in enumerate(new_spikes):
    new_waveform_start = max(s - window_size // 2, 0)
    new_waveform_end = min(s + window_size // 2, len(filtered_data))
    new_waveform = filtered_data[new_waveform_start:new_waveform_end]
    if len(new_waveform) < window_size:
        new_waveform = np.concatenate((np.zeros(window_size - len(new_waveform)), new_waveform))
    new_waveforms[i, :] = new_waveform

# Plot waveforms
plt.figure()
plt.plot(new_waveforms.T)
plt.xlabel('Time (samples)')
plt.ylabel('Amplitude (uV)')
plt.title('Waveforms of Detected Spikes')
plt.show()
new_waveforms = np.array(new_waveforms)
```

```
[ 845516 2350824 3253884 3643715 3684329 3691450 3691456 4107918 4393995
 4403675]
```



Using K-Means Clustering with New Threshold

```
# Apply PCA on the waveform matrix
pca = PCA(n_components=8)
new_waveform_pca = pca.fit_transform(new_waveforms)
print(pca.singular_values_)

```

```
new_PC1 = new_waveform_pca[:, 0]
new_PC2 = new_waveform_pca[:, 1]
new_PC3 = new_waveform_pca[:, 2]
new_PC = [new_PC1, new_PC2, new_PC3]

new_pca_waveform = new_PC

new_pca_waveform = np.array(new_PC)
new_pca_waveform = np.transpose(new_pca_waveform)
```

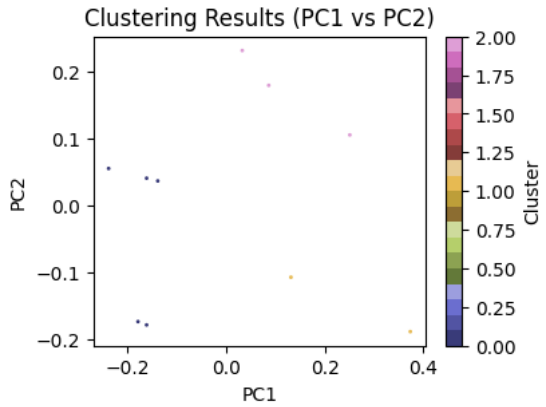
```
[0.62258675 0.45924231 0.25186375 0.14795406 0.0889995  0.07161821
 0.06647049 0.01176512]
```

```
# Perform K-Means clustering
n_clusters = 3  # Number of clusters
kmeans = KMeans(n_clusters=n_clusters, n_init="auto")
kmeans.fit(new_pca_waveform)  # Use PC1, PC2, and PC3 as features for clustering

# Get the cluster labels for each waveform
new_cluster_labels = kmeans.labels_

# Print the cluster labels
print("Cluster Labels:", new_cluster_labels)
```

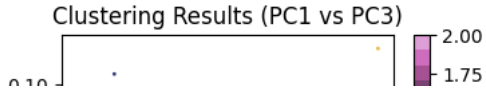
```
Cluster Labels: [0 0 0 0 2 0 2 1 2 1]
```

Scatter plot of #3 main component principle using new threshold

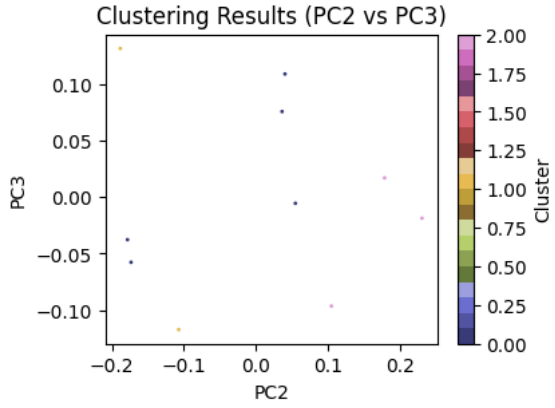
```
import matplotlib.pyplot as plt

# Scatter plot for PC1 and PC2
plt.figure(figsize=(4, 3))
plt.scatter(new_PC1, new_PC2, c=new_cluster_labels, cmap='tab20b', s=1)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('Clustering Results (PC1 vs PC2)')
plt.colorbar(label='Cluster')
plt.show()
```




```
# Scatter plot for PC1 and PC3
plt.figure(figsize=(4, 3))
plt.scatter(new_PC1, new_PC3, c=new_cluster_labels, cmap='tab20b', s=1)
plt.xlabel('PC1')
plt.ylabel('PC3')
plt.title('Clustering Results (PC1 vs PC3)')
plt.colorbar(label='Cluster')
plt.show()
```

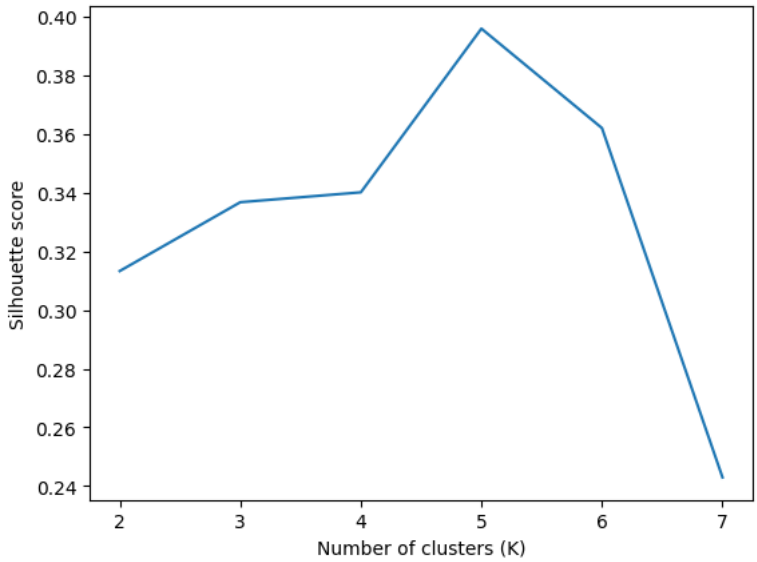



```
# Scatter plot for PC2 and PC3
plt.figure(figsize=(4, 3))
plt.scatter(new_PC2, new_PC3, c=new_cluster_labels, cmap='tab20b', s=1)
plt.xlabel('PC2')
plt.ylabel('PC3')
plt.title('Clustering Results (PC2 vs PC3)')
plt.colorbar(label='Cluster')
plt.show()
```



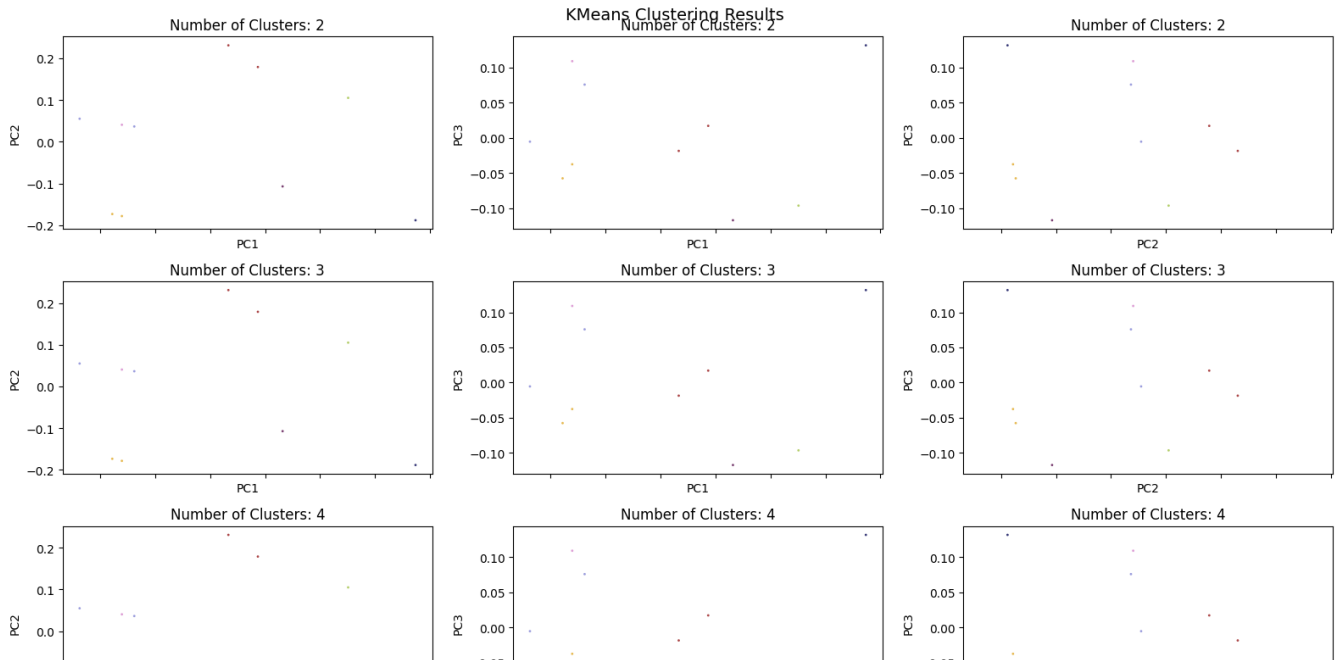
Using silhouette_score to Find a K which was the best performance using new threshold

```
# Define a range of values for K
k_values = range(2, 8)

# Calculate silhouette scores for each value of K
new_silhouette_scores = []
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
    new_labels = kmeans.fit_predict(new_waveform_pca)
    score = silhouette_score(new_waveform_pca, new_labels)
    new_silhouette_scores.append(score)

# Plot the silhouette scores for each value of K
plt.plot(k_values, new_silhouette_scores)
plt.xlabel('Number of clusters (K)')
plt.ylabel('Silhouette score')
plt.show()

```



Plot the Result of clustering with different K's with new threshold

```
# Repeat part (k) and (l) with different values of K for the clustering algorithm
k = 8
k_values = range(2, k)
fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
for i in k_values:
    kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(new_waveform_pca)
    labels = kmeans.labels_
    axes[i-2, 0].scatter(new_waveform_pca[:,0], new_waveform_pca[:,1], c = new_labels, s=1, cmap='tab20b')
    axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 0].set_xlabel('PC1')
    axes[i-2, 0].set_ylabel('PC2')

    axes[i-2, 1].scatter(new_waveform_pca[:,0], new_waveform_pca[:,2], c = new_labels, s=1, cmap='tab20b')
    axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 1].set_xlabel('PC1')
    axes[i-2, 1].set_ylabel('PC3')

    axes[i-2, 2].scatter(new_waveform_pca[:,1], new_waveform_pca[:,2], c = new_labels, s=1, cmap='tab20b')
    axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
    axes[i-2, 2].set_xlabel('PC2')
    axes[i-2, 2].set_ylabel('PC3')

# Add overall title and adjust layout
fig.suptitle('KMeans Clustering Results', fontsize=14)
fig.tight_layout()
```



Compute performance metrics of the K-means clustering for new threshold

```
# Compute performance metrics
time_window = 0.001  # +/- 1ms
true_positives = 0
false_positives = 0
false_negatives = 0

for spike_time in spike_idx:
    if np.any(np.abs(peaks - spike_time) <= time_window * fs):
        true_positives += 1
    else:
        false_negatives += 1

for spike_time in new_spikes:
    if np.all(np.abs(spike_time - spike_idx) > time_window * fs):
        false_positives += 1

precision = true_positives / (true_positives + false_positives)
recall = true_positives / (true_positives + false_negatives)
f1_score = 2 * (precision * recall) / (precision + recall)

print("Precision: %.3f" % precision)
print("Recall: %.3f" % recall)
print("F1-Score: %.3f" % f1_score)

```

```
Precision: 0.999
Recall: 0.957
F1-Score: 0.978
```

Feature extraction and Dimension Reduction Using t-SNE

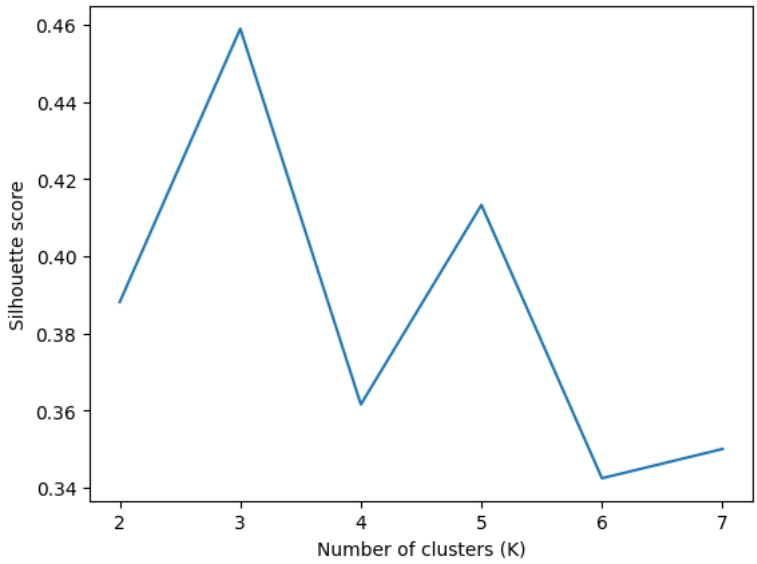
```
# Apply t-SNE to reduce dimensionality
tsne = TSNE(n_components=3, random_state=0)
tsne_feature = tsne.fit_transform(waveforms)
```

```
tsne_1 = tsne_feature[:, 0]
tsne_2 = tsne_feature[:, 1]
tsne_3 = tsne_feature[:, 2]
tsne_features = [tsne_1, tsne_2, tsne_3]

tsne_features = np.array(tsne_features)
tsne_features = np.transpose(tsne_features)
```

Using silhouette_score to Find a K which was the best performance for t-SNE algorithm

```
# Define a range of values for K
k_values = range(2, 8)
```

```
 3
 4 # Calculate silhouette scores for each value of K
 5 silhouette_scores = []
 6 for k in k_values:
 7     kmeans = KMeans(n_clusters=k, random_state=10, n_init="auto")
 8     labels = kmeans.fit_predict(tsne_features)
 9     score = silhouette_score(tsne_features, labels)
10     silhouette_scores.append(score)
11
12 # Plot the silhouette scores for each value of K
13 plt.plot(k_values, silhouette_scores)
14 plt.xlabel('Number of clusters (K)')
15 plt.ylabel('Silhouette score')
16 plt.show()
17
```



Plot the Result of clustering with different K's using t-SNE algorithm

```
 1 # Repeat part (k) and (l) with different values of K for the clustering algorithm
 2 k = 8
 3 k_values = range(2, k)
 4 fig, axes = plt.subplots(k-2, 3, sharex=True, figsize=(4 * 4, (k-2) * 3))
 5 for i in k_values:
 6     kmeans = KMeans(n_clusters=i, random_state=k, n_init="auto").fit(tsne_features)
 7     labels = kmeans.labels_
 8     axes[i-2, 0].scatter(tsne_feature[:,0], tsne_features[:,1], c = labels, s=1, cmap='tab20b')
 9     axes[i-2, 0].set_title('Number of Clusters: ' + str(i))
10
11     axes[i-2, 1].scatter(tsne_feature[:,0], tsne_features[:,2], c = labels, s=1, cmap='tab20b')
12     axes[i-2, 1].set_title('Number of Clusters: ' + str(i))
13
14     axes[i-2, 2].scatter(tsne_feature[:,1], tsne_features[:,2], c = labels, s=1, cmap='tab20b')
15     axes[i-2, 2].set_title('Number of Clusters: ' + str(i))
16
17
18 # Add overall title and adjust layout
19 fig.suptitle('KMeans Clustering Results', fontsize=14)
20 fig.tight_layout()
```